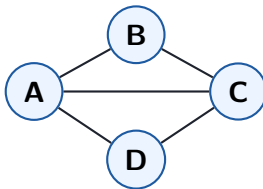


Graph Terminologies and Graph Representations

Dr. Md. Shamsuzzoha Bayzid

May 1, 2026



Learning Objectives

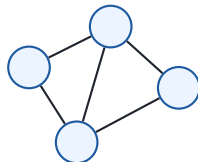
By the end of this lecture, students should be able to:

- define a graph using vertices and edges;
- identify common graph types: directed, undirected, weighted, simple, multigraph;
- explain degree, path, walk, trail, cycle, and connectivity;
- represent a graph using an adjacency list;
- represent a graph using an adjacency matrix;
- represent a graph using an incidence matrix;
- compare common graph representations in terms of space and operations.

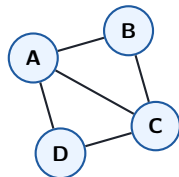
Motivation: Why Graphs Matter

Graphs are used whenever we need to model **relationships** between objects.

- people and friendships
- cities and roads
- web pages and hyperlinks
- computers and network links



Social network



Road map

Key idea

A graph stores objects and the connections among them.

What Is a Graph?

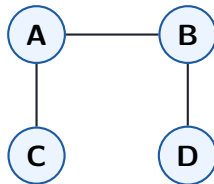
A graph is a structure consisting of:

- a set of **vertices** or **nodes**;
- a set of **edges** connecting pairs of vertices.

$$G = (V, E)$$

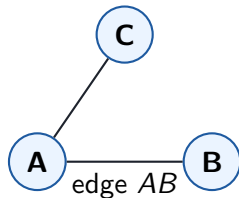
Example:

$$V = \{A, B, C, D\}, \quad E = \{AB, AC, BD\}$$



Vertices, Edges, Adjacency, Incidence

- A **vertex** is an object in the graph.
- An **edge** is a connection between two vertices.
- Two vertices are **adjacent** if an edge connects them.
- An edge is **incident** on its endpoints.

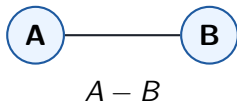


A and B are adjacent

Directed vs. Undirected Graphs

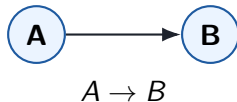
Undirected graph

Edges have no direction. Example: friendship.



Directed graph

Edges have direction. Example: following on social media.

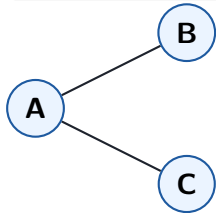


In a directed graph, (A, B) and (B, A) are different edges.

Weighted vs. Unweighted Graphs

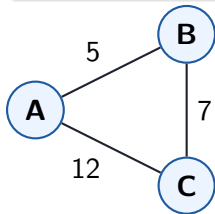
Unweighted graph

Edges only indicate whether two vertices are connected.



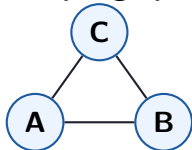
Weighted graph

Edges carry values: cost, distance, delay, capacity, etc.



Simple Graph, Multigraph, and Self-loop

Simple graph



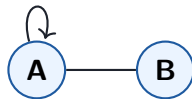
No loops, no repeated edges.

Multigraph



Multiple edges may connect the same pair.

Self-loop

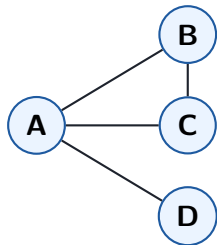


An edge can start and end at the same vertex.

Degree of a Vertex

For an undirected graph, the **degree** of a vertex is the number of edges incident on it.

$$\deg(v) = \text{number of edges touching } v$$



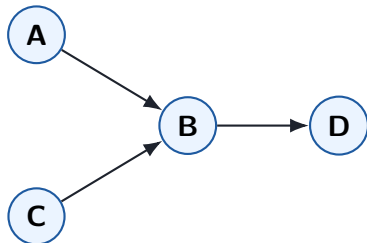
- $\deg(A) = 3$
- $\deg(B) = 2$
- $\deg(C) = 2$
- $\deg(D) = 1$

Note

In an undirected graph, a self-loop contributes 2 to the degree.

In-degree and Out-degree

For directed graphs, we distinguish incoming and outgoing edges.



Definitions

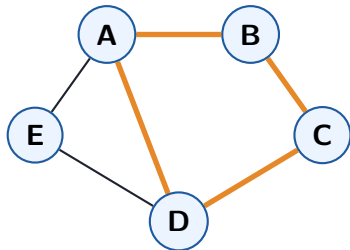
- $\deg^-(v)$: number of incoming edges
- $\deg^+(v)$: number of outgoing edges

For vertex B :

$$\deg^-(B) = 2, \quad \deg^+(B) = 1$$

Walk, Trail, Path, and Cycle

- **Walk:** vertices and edges may repeat.
- **Trail:** edges do not repeat.
- **Path:** vertices do not repeat.
- **Cycle:** starts and ends at the same vertex, with no repeated internal vertices.

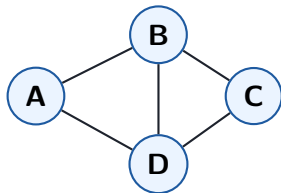


Example cycle

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Connected and Disconnected Graphs

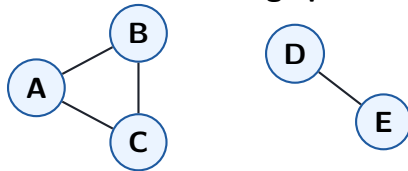
Connected graph



There is a path between every pair of vertices.

A maximal connected part is called a **connected component**.

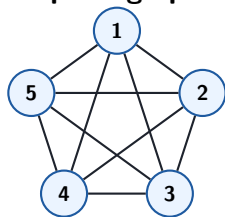
Disconnected graph



At least one pair of vertices has no path between them.

Complete, Sparse, and Dense Graphs

Complete graph K_5



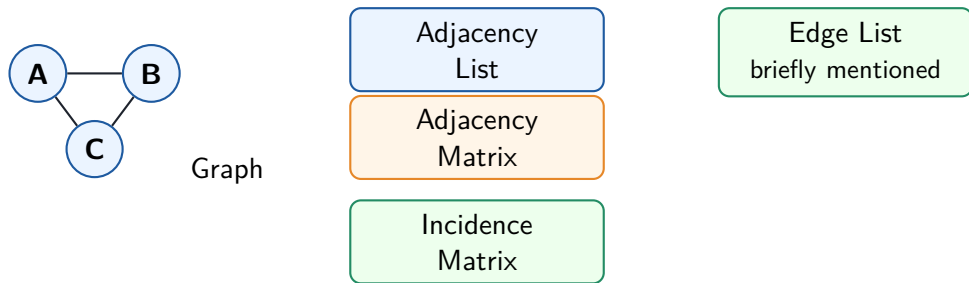
- A **complete graph** has an edge between every pair of distinct vertices.
- Notation: K_n .
- Number of edges in an undirected complete graph:

$$\frac{n(n-1)}{2}$$

- A **sparse graph** has relatively few edges.
- A **dense graph** has many edges.

Graph Representation Overview

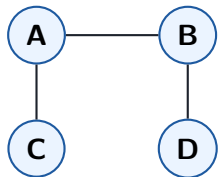
Computers cannot directly store graph drawings. We need data structures to store vertices and edges.



This lecture focuses on **adjacency list**, **adjacency matrix**, and **incidence matrix**.

Adjacency List Representation

In an adjacency list, each vertex stores a list of its neighboring vertices.



A: B, C

B: A, D

C: A

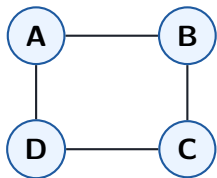
D: B

- For undirected graphs, each edge appears twice.
- For directed graphs, we usually store outgoing neighbors.

Adjacency List Example

Edges:

$(A, B), (A, D), (B, C), (C, D)$



Adjacency list:

A: B, D

B: A, C

C: B, D

D: A, C

Class questions:

- What are the neighbors of A?
- What is the degree of C?

Adjacency Matrix Representation

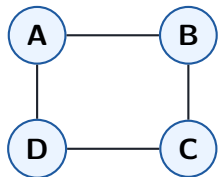
An adjacency matrix is a 2D array of size $|V| \times |V|$. For an unweighted graph:

$$M[i][j] = \begin{cases} 1, & \text{if there is an edge from } i \text{ to } j, \\ 0, & \text{otherwise.} \end{cases}$$

- Rows and columns represent vertices.
- For undirected graphs, the matrix is symmetric.
- For directed graphs, the matrix does not have to be symmetric.

Adjacency Matrix Example

Using the graph with edges (A, B) , (A, D) , (B, C) , (C, D) :



	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>A</i>	0	1	0	1
<i>B</i>	1	0	1	0
<i>C</i>	0	1	0	1
<i>D</i>	1	0	1	0

$$M[A][B] = 1 \text{ but } M[A][C] = 0.$$

Incidence Matrix Representation

In an **incidence matrix**, both vertices and edges are numbered.

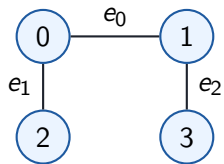
Suppose a graph has N vertices and M edges.

- Vertices are numbered $0, 1, \dots, N - 1$.
- Edges are numbered $0, 1, \dots, M - 1$.
- We use an $M \times N$ matrix: rows represent edges and columns represent vertices.
- For the row corresponding to an edge, we write 1 in the two columns of its endpoints; all other entries are 0.

Undirected incidence matrix

For an edge $e_i = (u, v)$, row i has 1 under column u and 1 under column v .

Incidence Matrix Example



Edges:

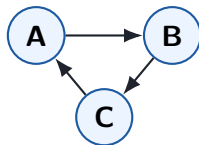
$$e_0 = (0, 1), \quad e_1 = (0, 2), \quad e_2 = (1, 3)$$

	0	1	2	3
e_0	1	1	0	0
e_1	1	0	1	0
e_2	0	1	0	1

Each row has exactly two 1s because each edge has two endpoints.

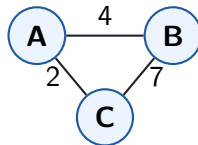
Directed and Weighted Matrices

Directed graph



	A	B	C
A	0	1	0
B	0	0	1
C	1	0	0

Weighted graph



	A	B	C
A	0	4	2
B	4	0	7
C	2	7	0

Comparing Graph Representations

Feature	Adjacency List	Adjacency Matrix	Incidence Matrix
Space	$O(V + E)$	$O(V^2)$	$O(EV)$
Rows represent	Vertices	Vertices	Edges
Columns represent	Neighbor entries	Vertices	Vertices
Best for	Sparse graphs, BFS/DFS	Dense graphs, fast edge checks	Studying edge-vertex relationships

Space Complexity Comparison

Let V be the number of vertices and E be the number of edges.

Adjacency list

$$O(V + E)$$

Good for sparse graphs.

Adjacency matrix

$$O(V^2)$$

Good for dense graphs.

Incidence matrix

$$O(EV)$$

Good for edge-vertex incidence.

Example: if $V = 1000$ and $E = 2000$, an adjacency matrix still needs

$$1000 \times 1000 = 1,000,000$$

entries.

Operation Costs

Operation	Adjacency List	Adjacency Matrix
Check whether (u, v) exists	$O(\deg(u))$	$O(1)$
Find all neighbors of u	$O(\deg(u))$	$O(V)$
Add an edge	Often $O(1)$	$O(1)$
Remove an edge	May require search	$O(1)$

Takeaway

Matrix is strong for edge lookup. List is strong for traversal.

Choosing the Right Representation

Use adjacency list when:

- the graph is sparse;
- you need to traverse neighbors often;
- you are implementing BFS or DFS.

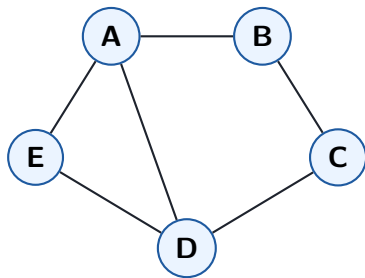
Use adjacency matrix when:

- the graph is dense;
- edge existence queries are frequent;
- the number of vertices is small or moderate.

Use an **incidence matrix** when the problem naturally focuses on which vertices are incident to which edges.

Example: a large social network is usually better stored using an adjacency list.

Classroom Activity: Convert This Graph



Work in pairs for 5–7 minutes.

- 1 Write the adjacency list.
- 2 Write the adjacency matrix.
- 3 Write the incidence matrix after numbering the edges.
- 4 Find the degree of each vertex.
- 5 Is the graph connected?
- 6 Identify one path and one cycle.

Summary

- A graph is $G = (V, E)$.
- Vertices represent objects; edges represent relationships.
- Graphs can be directed, undirected, weighted, unweighted, simple, or non-simple.
- Degree counts how many edges touch a vertex.
- Walks, trails, paths, and cycles describe movement inside a graph.
- Adjacency lists use $O(V + E)$ space.
- Adjacency matrices use $O(V^2)$ space but allow $O(1)$ edge lookup.
- Incidence matrices use rows for edges and columns for vertices.

Practice Problems / Homework

- 1 Define vertex, edge, degree, path, and cycle.
- 2 Draw an undirected graph with 5 vertices and 6 edges.
- 3 Convert your graph into an adjacency list.
- 4 Convert your graph into an adjacency matrix.
- 5 Number the edges and convert your graph into an incidence matrix.
- 6 For a directed graph, compute in-degree and out-degree of each vertex.
- 7 Compare adjacency list and adjacency matrix for a graph with 100 vertices and 200 edges.

Reflection question

Which representation would you choose for a sparse graph, and why?